

INFORME

Carrera: Analista de Sistemas

Materia: Aplicaciones Móviles

Alumno: Mateo Agustín Bonanata

INFORME TÉCNICO: EVALUACIÓN PARCIAL 2

Proyecto: CashFlow - Aplicación de Gestión Financiera Personal

1. Introducción y Arquitectura de la App

Propósito del Proyecto

"**CashFlow**" es una aplicación móvil nativa para la plataforma Android diseñada bajo el propósito de solucionar el control de finanzas de gastos personales. La herramienta proporciona a los usuarios la capacidad de llevar una contabilidad precisa e individual de sus transacciones cotidianas clasificadas por categorías, brindándoles un balance en tiempo real y una suite analítica que desglosa el destino de sus fondos monetarios.

Arquitectura de Navegación y Flujo de Pantallas

El sistema se estructura mediante un modelo de navegación multi-pantalla secuencial y aislado:

1. **Paso 1: Portal de Entrada (Autenticación)** Al iniciar la aplicación, el usuario interactúa con la LoginActivity. Esta pantalla funciona como un portal cerrado, o sea que no es posible saltarse este paso ni acceder a los datos financieros sin una sesión activa y validada de Firebase.
2. **Paso 2: Dashboard Principal (Consola Financiera)** Una vez autenticado exitosamente, el usuario es redirigido a la MainActivity, la cual opera como el panel

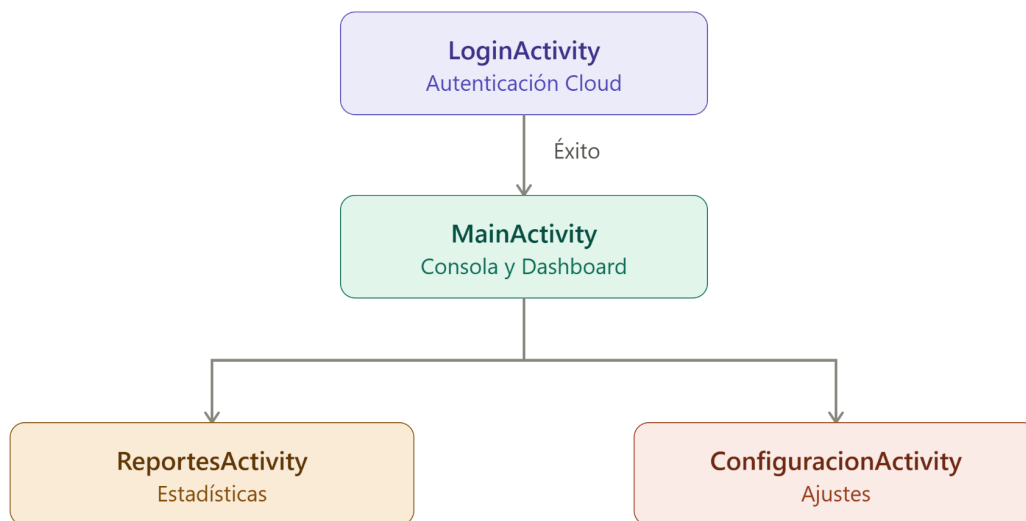
principal de control. Desde aquí se registran los nuevos consumos y se visualiza el saldo acumulado en tiempo real, junto a una lista cronológica completa de los últimos gastos realizados.

3. **Paso 3: Pantallas Secundarias (Ajustes y Reportes)** Desde el botón de ajustes que se encuentra en la esquina superior izquierda del Dashboard, el usuario puede desplazarse hacia:

- **Ajustes (ConfiguracionActivity):** Permite cambiar las configuraciones locales de visualización de perfil y apodo.

Mientras que desde el boton llamado Reportes, se accede a:

- **Reportes (ReportesActivity):** Muestra el análisis del estado financiero del usuario, agrupando los gastos por categoría y detallando cantidad.



2. Detalle Técnico y Funcional por Pantalla

A. Pantalla de Autenticación (LoginActivity)

Componentes de Interfaz (UI)

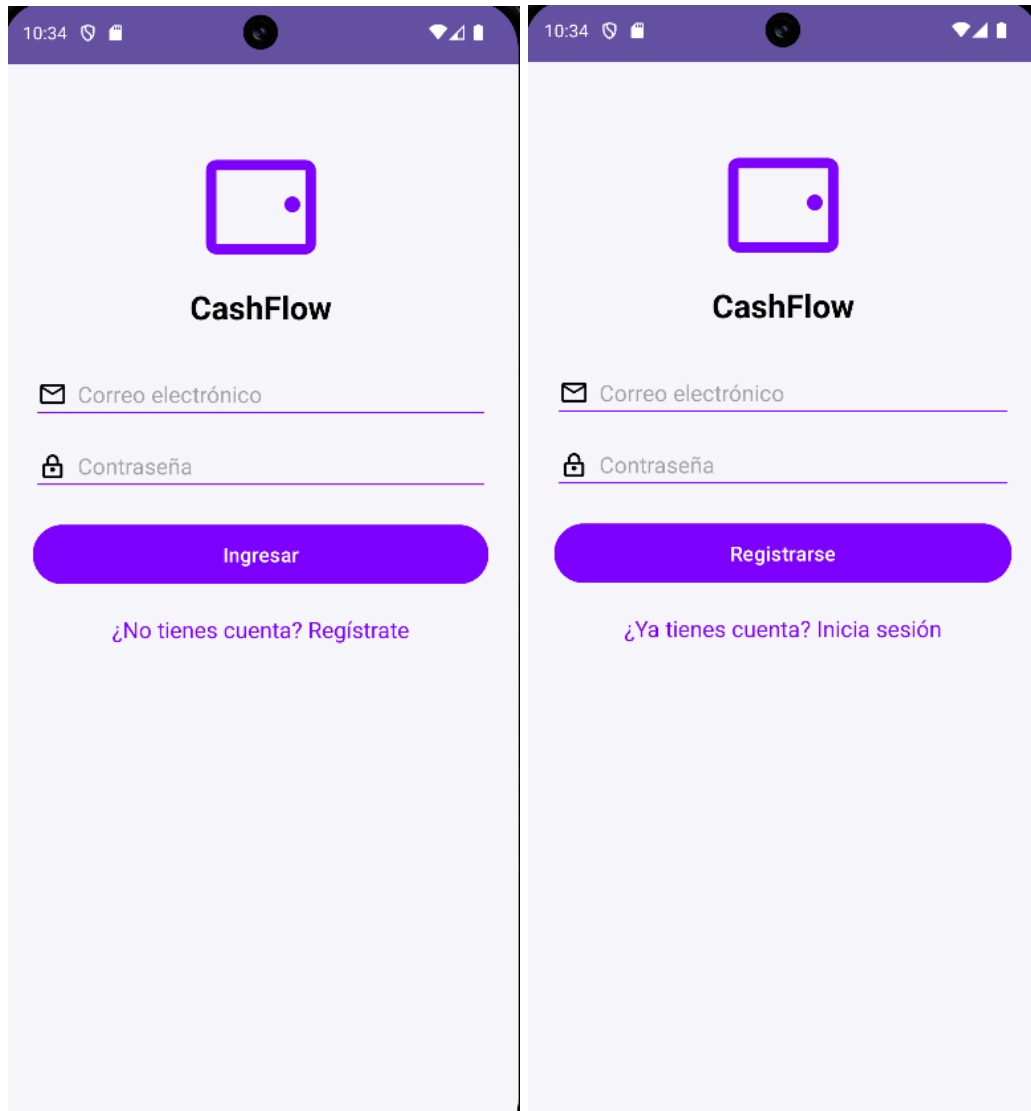
Diseñada utilizando un contenedor principal `ConstraintLayout` para asegurar un posicionamiento flexible y fluido en pantallas de diversos tamaños. Los campos de texto de correo (`EditText`) y contraseña utilizan elementos visuales estilizados con drawables vectoriales nativos para guiar al usuario.

Comportamiento Dinámico e Interacción Híbrida

Para optimizar la usabilidad del portal de entrada, se implementó una lógica híbrida reactiva en un solo archivo de layout (`activity_login.xml`). A través de una flag booleana (`isLoginMode`), la interfaz muta dinámicamente según la acción elegida por el usuario:

- **Modo Iniciar Sesión (Default):** El botón primario muestra "Ingresar" y el enlace de registro muestra "¿No tienes cuenta? Regístrate". Al hacer clic, se ejecuta la autenticación asíncrona mediante el método `signInWithEmailAndPassword` de `Firebase Authentication`.
- **Modo Registro:** Si el usuario presiona el enlace inferior, la bandera cambia, modificando el texto del botón principal a "Registrarse" y el enlace inferior a "¿Ya tienes cuenta? Inicia sesión". Al hacer clic en este modo, se ejecuta `createUserWithEmailAndPassword`.

Esta estrategia evita la duplicidad innecesaria de layouts y optimiza el flujo de la experiencia del usuario (UX) al no tener que cambiar de pantalla física para registrar una nueva cuenta.



B. Dashboard Principal (MainActivity)

Componentes de Interfaz (UI)

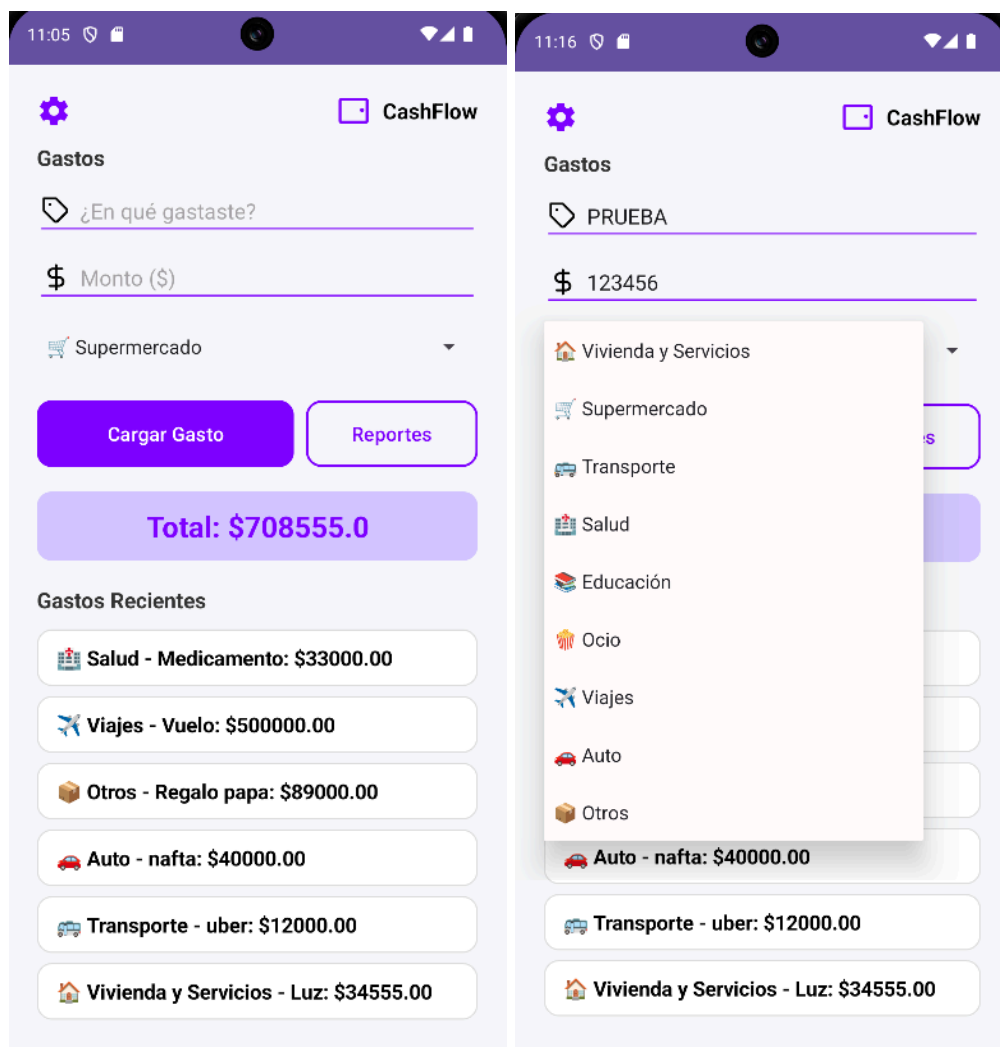
El Dashboard se compone de:

- Una barra superior minimalista con accesos al perfil y ajustes.
- Un formulario integrado de carga compuesto por un EditText para la descripción del gasto (acompañado por un icono de etiqueta de compra `ic_person`), un EditText numérico para el monto (acompañado del icono estilizado `ic_money`) y un Spinner nativo que consume dinámicamente un array de recursos XML de categorías (`categorias_array`)

- Los botones de acción rápida alineados en paralelo a través de un LinearLayout horizontal.
- Una sección inferior scroleable mediante un ScrollView que alberga de forma dinámica el containerGastos (un LinearLayout vertical).

Lógica de Negocio y Carga Reactiva

Al presionar el botón "Cargar Gasto" (btn_guardar), el controlador obtiene el texto seleccionado en el Spinner y los datos del formulario, aplicando validaciones de campos vacíos y formatos numéricos correctos. La visualización funciona a través de un Listener de Escucha en tiempo real de Firestore (addSnapshotListener). Cuando ocurre cualquier cambio en la base de datos (sea por inserción o eliminación de transacciones), se dispara de forma automática la actualización del total monetario acumulado y se regenera por completo la lista vertical cronológica, ordenando los gastos de forma descendente en base a su marca de tiempo (timestamp) procesada en memoria local en Java para evitar la necesidad de configurar índices compuestos en la nube.



C. Pantalla de Reportes (ReportesActivity)

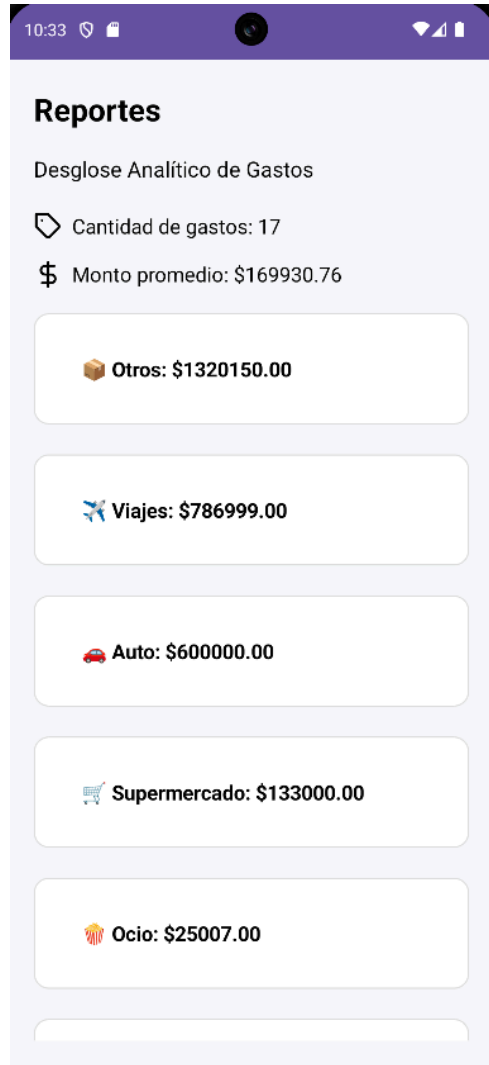
Componentes de Interfaz (UI)

Presenta una cabecera con tarjetas de métricas para la visualización del consolidado numérico de gastos (Cantidad de transacciones y Promedio monetario de consumo) con iconos vectoriales integrados. En la sección inferior, se despliega una matriz de consolidación financiera estructurada dinámicamente mediante un ScrollView y un LinearLayout vertical.

Consolidación Analítica con Estructuras de Datos

La pantalla lee de forma asíncrona la colección de gastos filtrando los documentos pertenecientes al usuario actual. Para estructurar la matriz de consolidación financiera, el controlador ejecuta la siguiente lógica en Java:

1. **Agrupación:** Itera a través de todos los documentos recuperados y acumula los montos en un HashMap<String, Double> asociando el String de la categoría (ej: "🛒 Supermercado") con el total monetario acumulado. Si el gasto no contiene categoría registrada (por ejemplo, transacciones antiguas), se consolida bajo el identificador de reserva "📦 Otros".
2. **Ordenamiento:** Transfiere las entradas del mapa a una lista estructurada (ArrayList) de entradas y aplica la interfaz Comparator para ordenar las categorías de manera descendente en base al monto gastado (las categorías de mayor impacto financiero aparecen primero).
3. **Inflación Dinámica:** Genera en tiempo de ejecución tarjetas visuales de texto (TextView) estilizadas únicamente para aquellas categorías cuyas acumulaciones superen el monto \$0.



D. Pantalla de Ajustes (ConfiguracionActivity)

Componentes de Interfaz (UI)

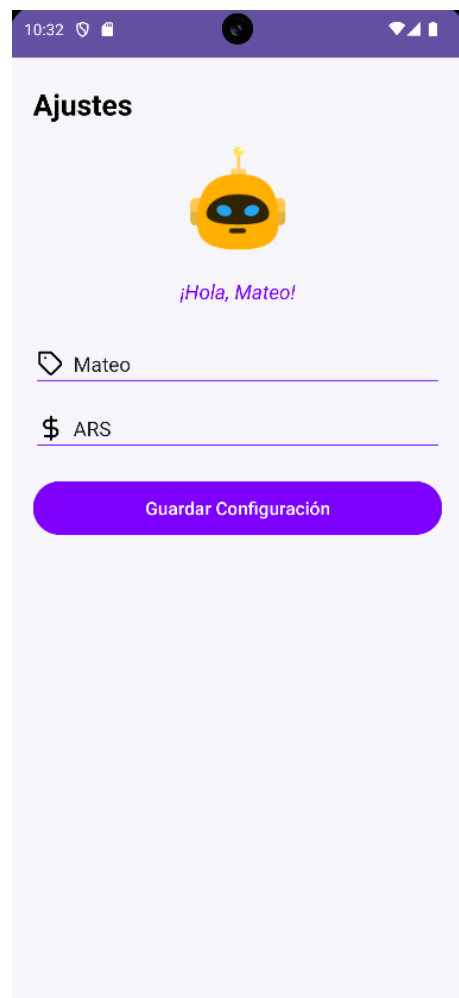
Diseñada de manera limpia con un contenedor central de configuraciones. En la parte superior, se destaca un ImageView circular destinado a hospedar el avatar digital del usuario, seguido de un texto de saludo dinámico y campos de texto para configurar preferencias de moneda e identidad.

Persistencia Local y Descarga Remota de Imágenes (Glide)

- **Persistencia Local Aislada:** La información referente al apodo y divisa preferida se resguarda físicamente en el dispositivo mediante el sistema de SharedPreferences. A fines de prevenir filtraciones entre diferentes cuentas que utilicen el mismo dispositivo

físico, las llaves de guardado están condicionadas y prefijadas dinámicamente con el UID de Firebase del usuario activo (ej: `currentUid + "_user_apodo"`).

- **Lógica de Fallback Dinámico:** Si un usuario abre la pantalla por primera vez y no cuenta con apodo registrado, el controlador extrae su correo electrónico desde el objeto `FirebaseUser`, segmenta el texto previo al símbolo `@` en Java, y lo aplica como nombre predeterminado de bienvenida temporal.
- **Descarga de Red con Glide:** Se incorporó la librería `Glide` para descargar de forma asíncrona y eficiente una imagen de perfil única a partir de una API de avatares públicos configurada dinámicamente con el UID del usuario ([https://api.dicebear.com/7.x/bottts/png?seed= + UID](https://api.dicebear.com/7.x/bottts/png?seed=)), logrando una identificación visual personalizada en la app sin consumir almacenamiento en la base de datos.



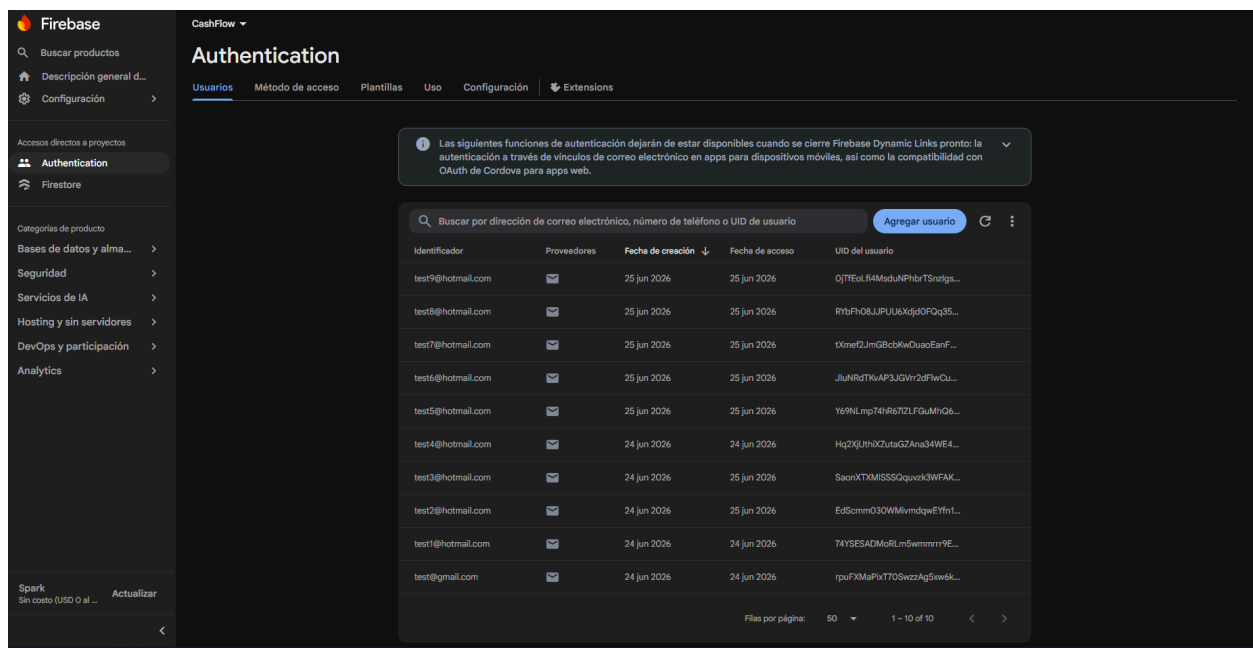
3. Comportamiento Dinámico e Interacciones Avanzadas

Aislamiento de Datos por UID en Firestore

La seguridad y el aislamiento de datos a nivel multiusuario en la nube se aseguran condicionando tanto las escrituras como las consultas a través de la API de Firebase Authentication. Cuando el usuario crea una transacción, se inyecta su clave de usuario única de forma mandatoria:

```
gasto.put("userId", currentUserId);
```

Posteriormente, al poblar el Dashboard o los Reportes, la consulta a Firestore aplica una restricción estricta mediante el método `.whereEqualTo("userId", currentUserId)`. Esto asegura que bajo ningún contexto un usuario pueda interceptar o visualizar los gastos de otra cuenta registrada en el sistema.



The screenshot shows the Firebase Authentication console for a project named 'CashFlow'. The 'Usuarios' (Users) tab is selected, displaying a table of registered users. A notification at the top indicates that certain authentication features will be deprecated when Firebase Dynamic Links is closed. The table lists 12 test users, all created on June 24 or 25, 2026, and accessed on the same dates. Each user has a unique UID and is associated with a 'test@' email address.

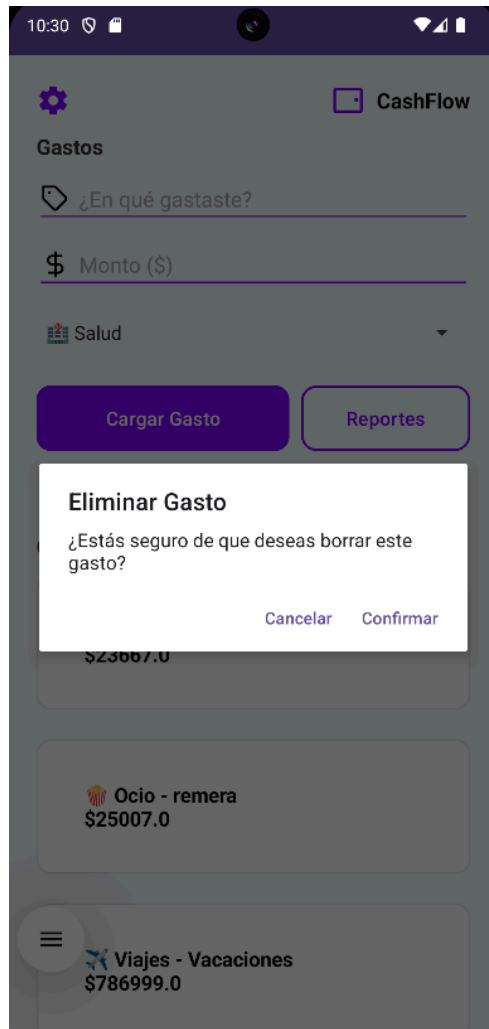
Identificador	Proveedores	Fecha de creación	Fecha de acceso	UID del usuario
test9@hotmail.com	📧	25 jun 2026	25 jun 2026	QjTEoLl4MaduAPhbrT5ndfg...
test8@hotmail.com	📧	25 jun 2026	25 jun 2026	RybFr08JJPUU6Xjd0FQq3S...
test7@hotmail.com	📧	25 jun 2026	25 jun 2026	TXmet2JmG8cbKwDuoEanf...
test6@hotmail.com	📧	25 jun 2026	25 jun 2026	JluNRdTKwAP3JGVrr2dFwCu...
test5@hotmail.com	📧	25 jun 2026	25 jun 2026	Y69NLmp74Hr67ZLFaMh06...
test4@hotmail.com	📧	24 jun 2026	24 jun 2026	Hq2XJlth0xZutaG2Ana34WE4...
test3@hotmail.com	📧	24 jun 2026	25 jun 2026	SaonXTXMISSQquock3WFAK...
test2@hotmail.com	📧	24 jun 2026	25 jun 2026	EdScmmO3OWMivmdqweYfn1...
test1@hotmail.com	📧	24 jun 2026	24 jun 2026	74YSESADMoRlm5wmmmr9E...
test@gmail.com	📧	24 jun 2026	24 jun 2026	rpufXMaPiX7T0SwezzAg5wek...

Eliminación Reactiva de Transacciones (Long Click)

Para simplificar la interfaz de usuario eliminando componentes de botones rígidos por cada transacción visualizada, se implementó un mecanismo gestual interactivo:

1. **Activación:** Cada tarjeta de gasto generada dinámicamente en el Dashboard cuenta con un Listener de interacción prolongada (`setOnLongClickListener`).

2. **Confirmación de Seguridad:** Al presionar de forma sostenida un elemento, se despliega un diálogo emergente nativo (AlertDialog.Builder) que consulta al usuario si desea eliminar la transacción seleccionada, evitando toques accidentales.
3. **Borrado en la Nube:** Si el usuario confirma, el sistema invoca el método `.delete()` directamente sobre el ID del documento en la colección de Firestore. La reactividad de la app se encarga de redibujar la pantalla al instante de forma automática al recibir la confirmación de la base de datos cloud.



4. Control de Versiones y Trabajo Incremental

Flujo de Desarrollo Continuo (Trunk-Based Development)

Para la ejecución de este proyecto se optó por una estrategia de desarrollo basada en la rama principal (main). Al tratarse de un desarrollo individual, este enfoque permitió mantener una integración continua de las funciones sin introducir la sobrecarga de gestión de ramas innecesarias, priorizando la agilidad.

Convención de Commits (Conventional Commits)

La prolijidad y legibilidad del repositorio se aseguraron mediante el cumplimiento estricto del estándar de Conventional Commits. Cada registro en el historial describe con exactitud la naturaleza del cambio y el módulo afectado:

- feat (Funcionalidades): Inserción de nuevas características operativas o de interfaz.
 - Ejemplo real: feat(logic): implementar categorización, eliminacion reactiva y agrupamiento analitico en reportes
 - Ejemplo real: feat(storage): agregar dependencia de Glide para carga de imágenes remota
- style (Estilos y UX): Ajustes de diseño visual, pulido de interfaces o layouts que no alteran la lógica de negocio.
 - Ejemplo real: style(ui): oscurecer fondo de banner de total y ajustar padding vertical de tarjetas
- refactor (Refactorización): Optimización o reorganización del código fuente existente sin modificar su comportamiento externo.
 - Ejemplo real: refactor(ui): extraer márgenes y tamaños fijos sp/dp de layouts a dimens.xml

LINK REPOSITORIO DE GITHUB:

<https://github.com/MateoDeIFlow/parcial-2-am-acn4b-bonanata>